Exercițiul 5 — Factory Method: cine decide ce clasă se naște

Proiect `design-patterns-java` · pachetul `app.ex5`

Context

Ai linii de text în care fiecare linie începe cu un tip:

```
STUDENT,Ana,Popescu,anul 2
PROFESOR,Ion,Ionescu,Matematica
STUDENT,Vlad,Marin,anul 1
```

Pentru fiecare linie construiești obiectul potrivit — `Student` sau `Profesor` — și le ții pe toate într-o singură `List<Utilizator>`. Ordinea câmpurilor: `TIP,Prenume,Nume,Detaliu`.

Prima variantă e un `switch` pe `campuri[0]` care se termină în `new`. Merge, și e aceeași durere de la `ex1`: ca să adaugi `ADMIN`, redeschizi o metodă care mergea. **Dar Strategy nu te salvează**: `new User(line)` din `UserRepository`-ul tău fixează tipul înainte ca polimorfismul să apuce să spună ceva. Recitește lecția 3 din `TEORIE.pdf` dacă nu-ți e clar de ce.

Factory Method: dacă `new Student(...)` nu poate fi ales polimorfic, ascunzi `new`-ul într-un obiect care POATE fi ales polimorfic — o fabrică.

Contractele

```
public abstract class Utilizator {
    public String getPrenume()
    public String getNume()
    public abstract String descriere();
}
```

```
public interface FabricaUtilizator {
    String getTip();
    Utilizator creeaza(String[] campuri);
}
```

Ce construiești

Clasa	Ce face
`Student`	`descriere()` → `[STUDENT] Ana Popescu – anul 2`
`Profesor`	`descriere()` → `[PROFESOR] Ion Ionescu – Matematica`
`FabricaStudent`	`getTip()` = `"STUDENT"` (gata); `creeaza` întoarce un `Student` din `campuri[1]`, `campuri[2]`, `campuri[3]`
`FabricaProfesor`	`getTip()` = `"PROFESOR"` (gata); `creeaza` întoarce un `Profesor`
`IncarcatorUtilizatori`	`incarca(List<String> linii)` → `List<Utilizator>`, folosind fabricile primite în constructor

Pașii (în ordinea asta — primul pas e o capcană intenționată)

1. **Simte durerea întâi.** Scrie `incarca` cu `switch (campuri[0])` și `new Student(...)` / `new Profesor(...)` direct. Rulează, vezi că merge. Apoi răspunde pe o foaie: dacă mâine apare `ADMIN`, ce fișiere atingi? Și mai important: **de ce nu poți scăpa de `switch` cu o metodă suprascrisă?**
2. **Mută fiecare `new` în fabrica lui.** `FabricaStudent.creeaza` are un singur `new Student(...)` și nimic altceva. La fel `FabricaProfesor`.
3. **Șterge `switch`-ul din încărcător.** Pentru fiecare linie: `split(",")`, apoi parcurgi fabricile și o întrebi pe fiecare dacă `getTip()` e egal cu `campuri[0]`. Când se potrivește, ceri produsul cu `creeaza(campuri)`.

Dacă niciuna nu se potrivește → `IllegalArgumentException("Unknown user type: " + campuri[0])`.

4. **Verifică.** Caută cuvântul `new` în `IncarcatorUtilizatori`. Dacă apare `Student` sau `Profesor` acolo, pasul 3 nu e terminat.
5. **Proba open/closed.** Adaugă tipul `ADMIN` scriind **doar** două clase noi: `Admin` (extinde `Utilizator`, câmp `departament`, `descriere()` → `[ADMIN] Alex Grozavu – IT`) și `FabricaAdmin` (`getTip()` = `"ADMIN"`). Nu există în starter: le scrii după modelul `Student` / `FabricaStudent`.

Ce găsești deja în proiect

Fișier	Ce conține
`Utilizator`	abstractă, gata: prenume, nume, constructorul cu validare, getter-ele, `descriere()` abstractă
`Student`, `Profesor`	câmpul în plus (`an` / `catedra`), constructorul cu validare și getter-ul gata; `descriere()` aruncă `UnsupportedOperationException`
`FabricaUtilizator`	contractul, gata
`FabricaStudent`, `FabricaProfesor`	`getTip()` gata; `creeaza` aruncă `UnsupportedOperationException`
`IncarcatorUtilizatori`	câmpul `fabrici` și constructorul cu validare gata; `incarca` aruncă `UnsupportedOperationException`
`Testare5`	`ruleaza()` gol

Cerințe

`Testare5.ruleaza()` exact așa (după pasul 5):

```
public static void ruleaza() {
    List<FabricaUtilizator> fabrici = List.of(
            new FabricaStudent(),
            new FabricaProfesor(),
            new FabricaAdmin());
    IncarcatorUtilizatori incarcator = new IncarcatorUtilizatori(fabrici);

    List<String> linii = List.of(
            "STUDENT,Ana,Popescu,anul 2",
            "PROFESOR,Ion,Ionescu,Matematica",
            "STUDENT,Vlad,Marin,anul 1",
            "ADMIN,Alex,Grozavu,IT");

    for (Utilizator utilizator : incarcator.incarca(linii)) {
        System.out.println(utilizator.descriere());
    }

    try {
        incarcator.incarca(List.of("SECRETAR,Dan,Pop,Registratura"));
    } catch (IllegalArgumentException ex) {
        System.out.println(ex.getMessage());
    }
}
```

`List.of(...)` face o listă fixă, doar de citit — e destul pentru un test. Importă și `FabricaAdmin`.

Ieșirea așteptată, exact așa:

```
[STUDENT] Ana Popescu – anul 2
[PROFESOR] Ion Ionescu – Matematica
[STUDENT] Vlad Marin – anul 1
[ADMIN] Alex Grozavu – IT
Unknown user type: SECRETAR
```

Constrângeri

- `IncarcatorUtilizatori` NU conține `new Student`, `new Profesor`, `new Admin` și niciun `switch` / `if` pe tipul de utilizator. Singura comparație permisă e între două șiruri: `fabrica.getTip().equals(campuri[0])`.
- Fără `instanceof`, `getClass()`, fără cast. Utilizatorii se folosesc DOAR prin `descriere()`.
- Mesajele de eroare în engleză.

Gata când

Ieșirea e identică, iar pentru `ADMIN` n-ai modificat nicio linie din `IncarcatorUtilizatori`. Dacă a trebuit, pattern-ul nu e complet.

Bonus

- **Fabrica cu validare proprie.** Fă `FabricaStudent.creeaza` să arunce `IllegalArgumentException` dacă linia nu are exact 4 câmpuri. Observă unde a ajuns validarea: lângă `new`-ul pe care îl apără, nu împrăștiată prin încărcător.
- **Întrebare pentru online-school.** În `UserRepository.loadData()` ai `new User(line)`. Ce ar juca acolo rolul lui `campuri[0]` ? Uită-te la `users.txt` înainte să răspunzi.